

Resultados — Antipatrón del delay() con Múltiples LEDs

Sistemas Programables

1. Comportamiento esperado vs. comportamiento real

Si cada LED funcionara de manera verdaderamente independiente (como sugiere la intención del código), cada uno parpadearía con su propio periodo, de forma simultánea a los demás:

LED	Tiempo encendido	Tiempo apagado	Parpadeo esperado (independiente)
LED1	500 ms	500 ms	Cada 1000 ms (ciclo propio)
LED2	1000 ms	1000 ms	Cada 2000 ms (ciclo propio)
LED3	1500 ms	1500 ms	Cada 3000 ms (ciclo propio)

Sin embargo, debido al uso de delay(), el microcontrolador ejecuta cada bloque de forma secuencial y bloqueante. La siguiente tabla muestra el orden real de los eventos durante un ciclo completo del programa:

Paso	Evento observado	Tiempo transcurrido desde el inicio del ciclo
1	LED1 se enciende	0 ms
2	LED1 se apaga	500 ms
3	LED2 se enciende	1000 ms
4	LED2 se apaga	2000 ms
5	LED3 se enciende	3000 ms
6	LED3 se apaga	4500 ms

2. Duración total del ciclo

Sumando los tiempos de encendido y apagado de los tres LEDs, un solo ciclo completo del loop() tarda:

$500 + 500 + 1000 + 1000 + 1500 + 1500 = 6000 \text{ ms (6 segundos)}$

Esto quiere decir que, en la práctica, el LED1 no vuelve a parpadear cada 500 ms como se pretendía originalmente, sino que su siguiente parpadeo ocurre hasta que los otros dos LEDs terminen su propio ciclo, es decir, cada 6000 ms.

3. Observaciones

- Los tres LEDs se encienden y apagan uno a la vez, nunca simultáneamente entre sí.
- El tiempo real de repetición de cada LED corresponde a la suma de todos los delay() del programa (6000 ms), y no al tiempo individual programado para cada uno.

- Durante los `delay()`, el Arduino queda completamente inactivo para cualquier otra tarea; por ejemplo, si se agregara la lectura de un botón, esta no se detectaría hasta que terminara el `delay()` en curso.
- Visualmente, el patrón se percibe como una secuencia LED1 → LED2 → LED3 en lugar de un parpadeo independiente y simultáneo de los tres.

4. Conclusión de resultados

Los resultados confirman el comportamiento bloqueante descrito en el reporte: el uso de `delay()` para múltiples tiempos no produce parpadeos independientes, sino una cadena de esperas secuenciales que se acumulan. Esto valida la necesidad de utilizar una estrategia no bloqueante, como el control de tiempos mediante `millis()`, para lograr que cada LED efectivamente parpadee según su propio intervalo sin depender de los demás.